When designing a custom filter, the desired frequency response is defined by the values in an array. Now consider this: what does the frequency response do *between* the specified points? For simplicity, two cases can be imagined, one "good" and one "bad." In the "good" case, the frequency response is a smooth curve between the defined samples. In the "bad" case, there are wild fluctuations between. As luck would have it, the impulse response in (b) corresponds to the "bad" frequency response. This can be shown by padding it with a large number of zeros, and then taking the DFT. The frequency response obtained by this method will show the erratic behavior between the originally defined samples, and look just awful.

To understand this, imagine that we force the frequency response to be what we want by defining it at an infinite number of points between 0 and 0.5. That is, we create a continuous curve. The inverse DTFT is then used to find the impulse response, which will be *infinite* in length. In other words, the "good" frequency response corresponds to something that cannot be represented in a computer, an infinitely long impulse response. When we represent the frequency spectrum with $N/2 + 1$ samples, only N points are provided in the time domain, making it unable to correctly contain the signal. The result is that the infinitely long impulse response wraps up (aliases) into the N points. When this aliasing occurs, the frequency response changes from "good" to "bad." Fortunately, windowing the N point impulse response greatly reduces this aliasing, providing a smooth curve between the frequency domain samples.

Designing a digital filter to produce a given frequency response is quite simple. The hard part is finding what frequency response to use. Let's look at some strategies used in DSP to design custom filters.

Deconvolution

Unwanted convolution is an inherent problem in transferring analog information. For instance, all of the following can be modeled as a convolution: image blurring in a shaky camera, echoes in long distance telephone calls, the finite bandwidth of analog sensors and electronics, etc. Deconvolution is the process of filtering a signal to compensate for an undesired convolution. The goal of deconvolution is to recreate the signal as it existed *before* the convolution took place. This usually requires the characteristics of the convolution (i.e., the impulse or frequency response) to be known. This can be distinguished from **blind deconvolution**, where the characteristics of the parasitic convolution are *not* known. Blind deconvolution is a much more difficult problem that has no general solution, and the approach must be tailored to the particular application.

Deconvolution is nearly impossible to understand in the *time domain*, but quite straightforward in the *frequency domain*. Each sinusoid that composes the original signal can be changed in amplitude and/or phase as it passes through the undesired convolution. To extract the original signal, the deconvolution filter must *undo* these amplitude and phase changes. For